A retryable batch data system for municipal air-quality readings

Project purpose and users

This project designs a portable ingestion system that lets a municipality use environmental sensor readings for planning and public information. Its operational purpose is to keep historical observations consistently accessible by station and time. Planners compare locations over a chosen period, analysts inspect pollutant and weather values, and developers feed dashboards or a separate alert service from the stored data. The prototype stops at reliable storage: health thresholds and warnings need municipal policy, sensor-quality rules and an accountable operating process.

Dataset and data model

The sample is the UCI Beijing Multi-Site Air Quality dataset with 420,768 hourly records from 12 stations (2013–2017). Each row has station and time parts, pollutants such as PM2.5, PM10, SO_2, NO_2, CO and O_3, and weather values such as temperature, pressure, dew point, rain and wind (UCI Machine Learning Repository, n.d.). Its size is close to the half-million-record scenario, and its varied fields allow me to model evolving sensors. One MongoDB document represents one station-hour: a stable outer structure holds station, timestamp, source timezone and source, and a nested measurements object holds the readings. Numbers stay numeric, missing values are omitted, unknown columns are kept, and the source time is read as Asia/Shanghai and stored explicitly so that the assumption stays visible.

Architecture and design choices

Docker Compose starts MongoDB and a Python ingestion container. The loader waits for the database health check, downloads the public archive, streams the CSV files, transforms rows and writes batches of 1,000, so the full dataset is never held in memory. An initialisation script validates the stable fields and indexes station plus timestamp. MongoDB fits because a document maps naturally to a reading and new sensor metrics need no redesign of existing records, while validation still guards the shared fields (MongoDB, n.d.-a). PostgreSQL would be preferable for a fixed schema or join-centred queries, and CSV alone offers no indexed, concurrent query service. A larger analytical platform could later serve historical aggregates but would add cost and setup now.

Recovery and growth

The loader derives each document ID from station and observation time and uses unordered upserts with `$setOnInsert`. Replaying a batch after an uncertain write therefore creates no duplicates (MongoDB, n.d.-c). Transient database errors trigger bounded exponential backoff, and an interrupted run can simply start again: stored records stay and missing ones are added. This suits a static sample; a continuously changing source would need durable checkpoints, source versioning, a correction policy, malformed-row handling and monitoring.

The Compose setup shows portability but is a single container, not a highly available system. A larger deployment could use a replica set for redundancy and, if load justified it, a sharded cluster,

which spreads data over machines at the price of more infrastructure (MongoDB, n.d.-b). Production would also require protected secrets, TLS, backups, access control, monitoring and performance tests. The shard key should follow real dashboard queries; cloud hosting alone does not guarantee scalability.

Results and personal reflection

The repository (github.com/yerkebttt/municipal-air-quality-batch-ingestion) contains the loader, container definitions, initialisation script, README and unit tests for normalisation, time zones, missing values, an unknown CO_2 field, batching and invalid batch sizes.

I learned that recovery rests on safe repeated writes, not on exception handling alone. A deterministic key with an insert-only upsert answers the question of uncertain batch results, and a stable outer document with flexible measurements answers unknown future sensors. The main problems were unclear future sensor structure, ambiguous local versus UTC time and the risk of duplicates on retry; I solved them with a flexible nested object, an explicit timezone field and idempotent writes. In similar projects I will settle time-zone and correction rules first, test representative transformations early and verify the complete deployment before adding downstream features.

References

UCI Machine Learning Repository. (n.d.). *Beijing Multi-Site Air Quality*. https://archive.ics.uci.edu/dataset/501/beijing+multi+site+air+quality+data

MongoDB. (n.d.-a). *Data modeling*. https://www.mongodb.com/docs/manual/data-modeling/

MongoDB. (n.d.-b). *Sharding*. https://www.mongodb.com/docs/manual/sharding/

MongoDB. (n.d.-c). *PyMongo bulk write operations*. https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/bulk-write/